Principles & Practices — Leave Management System

Team: Kamalasaravanan, Harish | github.com/ee24s063/Leave_Management_System | October 2026

About the project. A web app where employees apply for leave, Team Leads and HR approve it, and HR tracks balances on a dashboard and downloads Excel reports. Built with Java Spring Boot and an SQLite database. We planned the work in phases and used an AI coding assistant (Claude Code) to write code; we made the decisions and checked every result.

1. Prompt Engineering

We write every prompt like a small, clear instruction. Our rules:

- **Give context.** We point to our project plan ("continue with Phase 4") instead of explaining everything again.
- **One task at a time.** One phase or one change per prompt ("complete Phase 5 first"), so each result is easy to check.
- **Decide once.** Our decisions (e.g. "stay with SQLite") are written in the plan, so they are not changed later by mistake.
- **Check risks before big changes.** We ask what could go wrong ("if I switch to JWT login, what will be the issue?"), then we decide.
- **Say when it is done.** We state the finish line and ask to see results first ("run the tests and share the results with me before committing").

2. Debugging

For every bug we follow the same steps: **1. Reproduce it** → **2. Collect evidence** (logs, screenshots, test results) → **3. Find the real cause** → **4. Make the smallest fix** → **5. Add a test** and run all tests again. Examples:

Problem	How we found the cause	Fix
Forms stopped working after the login change	Turned on detailed security logs	A security token changed on every page; we made it stay the same
Table headings were invisible	A screenshot from the UI test	Changed the heading colour
A list sometimes showed the wrong tab's data	UI test failure and screenshot	Ignore late data from a tab the user has left

3. Version Control (Git)

- **All code is on GitHub.** We commit only when all tests pass and we have reviewed the results.
- **Database changes are saved as code** (numbered migration files V1–V4), so anyone can rebuild the same database.
- **Private data is never committed:** the database file and build files are excluded with `.gitignore`.
- **Prompts are saved in Git too**, in `docs/prompts/PROMPT_LOG.md`: each prompt, its purpose, the result and the commit it belongs to.
- **One branch per feature**, merged into `main` through a pull request that the other team member reviews.

4. Testing, Refactoring & Documentation

Testing: 245 automated tests run with one command (`mvn test`) in about a minute, and all of them pass.

Type	What it checks	Tests
Unit tests (JUnit, Mockito)	Each part of the code on its own	141
Integration tests (Spring Boot)	The full app with a real database	91
UI tests (Selenium)	A real browser clicking through the pages as Employee, Team Lead and HR	13

- **Refactoring:** we improve code in small steps and run all tests after each step. Example: one shared Excel writer is used by all five reports.
- **Documentation:** the README is updated after every phase (features, setup, how to use the API). We also keep a project plan and a test report.

Appendix — Sample Prompts

Instructions we gave during the project (spelling tidied; exact wording in `docs/prompts/PROMPT_LOG.md`):

#	Our instruction	Rule it shows
1	No lower limit on leave balances; HR sees a warning but may still approve. Comp-off never expires. *(summarised from our first design decisions)*	Clear business rules
2	Stay with SQLite, don’t keep the current session login. Continue with Phase 4.	Our decision + one task
3	If I switch to JWT login, what will be the issue?	Check risks first
4	OK, switch to JWT. For the demo, localhost is enough.	Decision with limits
5	Complete Phase 5 first.	One task at a time
6	Before committing, generate JUnit tests for each module and run them. Also do UI tests. Share the test results in a document with me first.	Say when it is done
7	Commit the prompt log, and keep to feature branches.	Git workflow

Template we use for new work: *Context* (which phase, what is done) → *Task* (one change) → *Limits* (decisions to keep) → *Done when* (tests pass, README updated) → *Show me* (results before commit).